The benefits of agile integration, Part 2: Moving to a lightweight, agile integration architecture

A containerized and decentralized approach to cloud-ready connectivity

Kim J. Clark

May 01, 2018
(First published February 13, 2018)

Microservices principles are revolutionizing the way applications are built by enabling a more decoupled and decentralized approach to implementation, which leads to greater agility, scalability, and resilience. These applications still need to be connected to one another and to existing systems of record. It clearly makes sense to use microservice techniques in the integration space, too. Lightweight integration provides the benefits of cloud-ready containerization to integration architecture, and provides the opportunity to escape from the heavily centralized ESB pattern to more empowered and autonomous application teams.

This two-part series explores the approach that modern integration architectures are taking to ensure that they can be as agile as the applications they interconnect. The pattern of the centralized enterprise service bus (ESB) has served its purpose, and still has its place, but many enterprises are exploring a more containerized and decentralized approach to integration architecture. This approach is known as **agile integration architecture**, although note that the term *lightweight integration* has also been used in the past.

Part 1 explored the fate of the ESB. It looked at how and why the centralized ESB pattern arose in the era of service-oriented architecture (SOA), but also at the challenges that emerged as a result of it. It also considered where APIs came into the picture and what relationship there might be between all this and microservices architecture. This historical background will now enable us to make more confident statements about how to do things better in the future.

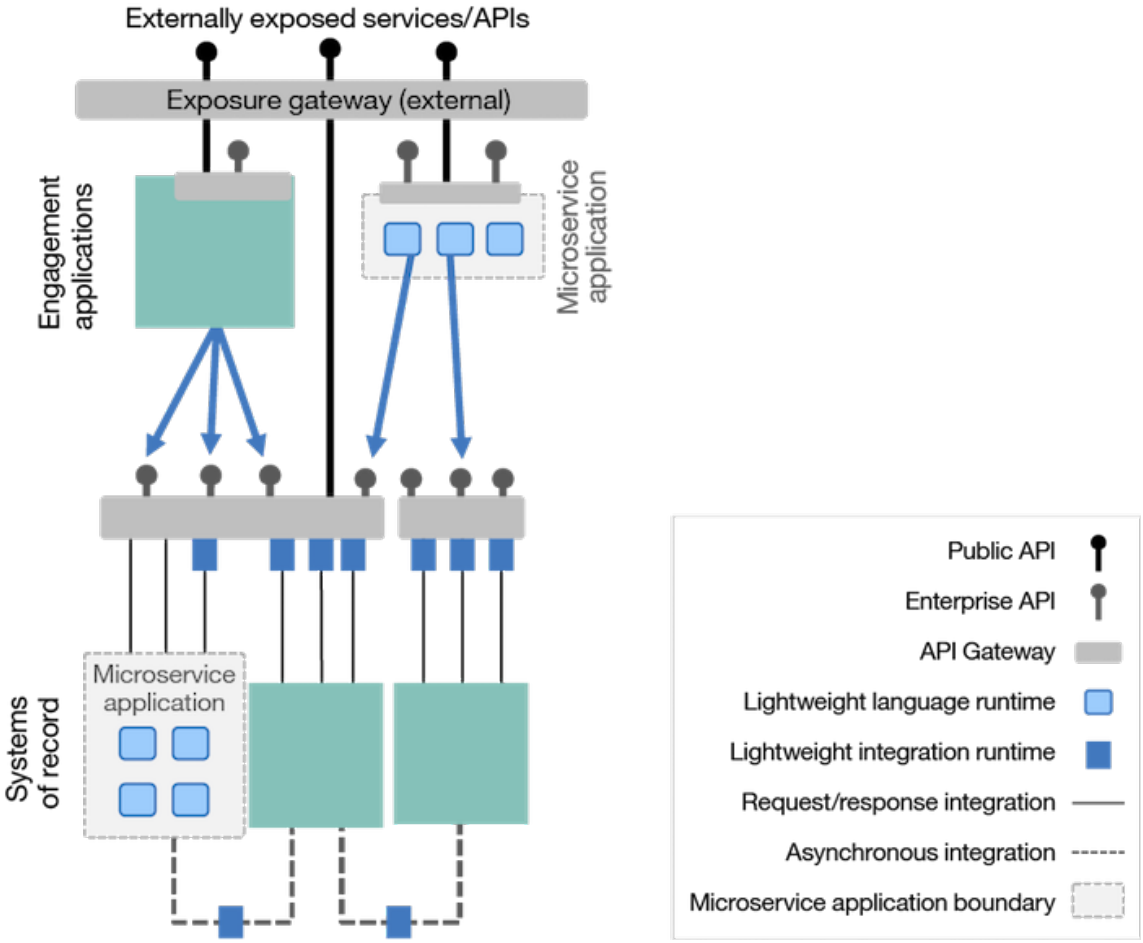
Part 2 will describe agile integration architecture, looking at how integration architecture can benefit from the technologies and principles behind microservices in order to ensure that new applications can perform the integration they need at the pace and scale of modern innovation. You will also see how integration can be more fundamentally decentralized in order to enable greater autonomy and productivity to lines of business. We'll begin by exploring the benefits of microservice principles in integration architecture.

© Copyright IBM Corporation 2018

Making use of microservice principles in integration

If it makes sense to build applications in a more granular fashion, why shouldn't we apply this idea to integration, too? We could break up the enterprise-wide centralized ESB component into smaller, more manageable, dedicated components. Perhaps even down to one integration runtime for each interface we expose, although in many cases it would be sufficient to bunch the integrations as a handful per component.

Figure 1. Breaking up the centralized ESB into independently maintainable and scalable pieces



The heavily centralized ESB pattern can be broken up in this way, and so can the older hub and spoke pattern. This makes each individual integration easier to change independently, and improves agility, scaling, and resilience.

This approach of fine-grained integration deployment allows you to make a change to an individual integration with complete confidence that you couldn't introduce any instability into the environment on which the other integrations are running. You could choose to use a different version of the integration runtime, perhaps to take advantage of new features, without forcing a risky upgrade to all other integrations. You could scale up one integration completely independently of the others, making extremely efficient use of infrastructure, especially when using cloud-based models.

Try IBM Cloud for free

Build your next app quickly and easily with IBM Cloud Lite. Your free account never expires, and you get 256 MB of Cloud Foundry runtime memory, plus 2 GB with Kubernetes Clusters. Get all the details and find out how to get started. And if you're new to IBM Cloud, check out the IBM Cloud Essentials course on developerWorks.

There are of course considerations to be worked through with this approach, such as the increased complexity with more moving parts. Also, although the above could be achieved using virtual machine technology, it is likely that the long-term benefits would be greater if you were to use containers such as Docker, and orchestration mechanisms such as Kubernetes. Introducing new technologies to the integration team can add a learning curve. However, these are the same challenges that an enterprise would already be facing if they were exploring microservices architecture in other areas, so that expertise may already exist within the organization.

The fine-grained integration deployment described above is just one aspect of what we more broadly describe as **agile integration architecture**. We use this term to differentiate it from more purist microservices application architecture. We also want to mark a distinction from the "ESB" term, which is strongly associated with the more cumbersome centralized integration architecture. It is worth noting that the term "lightweight integration" was used briefly for this concept, but "agile integration architecture" is now deemed to be more industry aligned.

We will go on to summarize other aspects of agile integration architecture in this article. A growing collection of more detailed articles on this topic are also available via a separate blog post.

What does a modern integration runtime look like?

Clearly, agile integration architecture requires that the integration topology be deployed very differently. A key aspect of that is a modern integration runtime that can be run in a container-based environment, and is well suited to cloud-native deployment techniques. Modern integration runtimes are almost unrecognizable from their historical peers. They are dependency-free, so they no longer require databases or message queues. They can be installed simply by laying them out on a file system and starting them up—ideal for the layered file systems of Docker images. All configuration is in properties files and command-line interfaces, so they can be easily incorporated into build pipelines. They are so lightweight that they can be started and stopped in seconds, and can be easily administered by orchestration frameworks such as Kubernetes.

IBM App Connect Enterprise (formerly known as IBM Integration Bus) is a good example of such a runtime, as described in more detail in this blog post. Like other integration servers, IBM App Connect Enterprise is not an ESB; ESB is just one of the patterns it can be used for. It is used in a variety of other architectural patterns too, and increasingly in the style of agile integration architecture.

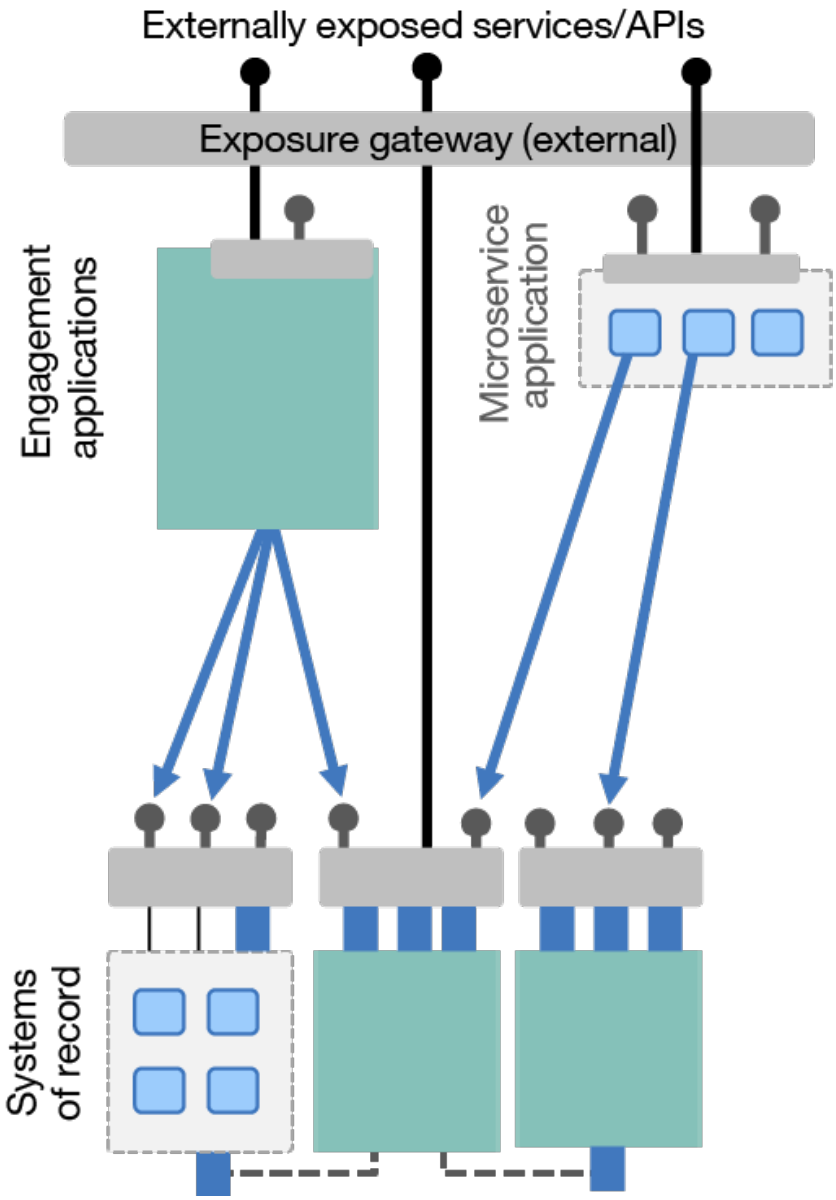
Decentralized integration ownership

We can take this a step further. Now that you've broken up the integrations into separate pieces, you can opt to distribute those pieces differently from an ownership and administration point of view. Note that not only have most integration runtimes become more lightweight, but they have

also become significantly easier to use. You no longer need to be a deep integration specialist to use a good modern integration runtime.

Technologically, there may be no difference between this and the preceding diagram. What's changed is who owns the integration components. Could you have the application teams take on integration themselves? Could they own the creation and maintenance of the integrations that belong to their applications? You'll notice we've also shown the decentralization of the exposure gateways to denote that the administration of the API's exposure also moves to the application teams.

Figure 2. Decentralizing integration to the application teams



There are many potential advantages to this decentralized integration ownership approach:

- A common challenge for separate SOA teams was that they didn't understand the applications they were exposing. The application teams know the data structures of their own applications better than anyone.
- Fewer teams will be involved in the end-to-end implementation of a solution, significantly reducing the cross-team chatter, latency, and inevitable waterfall development that typically occurs in these cases.

To re-iterate, decentralized integration is an organizational change, not a technical one.

Of course, this approach isn't right for every situation. It may work for some organizations, or for some parts of some organizations but not for others. Application teams for older applications may not have the right skill sets to take on the integration work. It may be that integration specialists need to be seeded into their team. This is a tool for potentially creating greater agility for change and scaling, but what if the application has been largely frozen for some time. At the end of the day, some organizations will find it more manageable to retain a more centralized integration team. The approach should be applied where the benefits are needed most. That said, this style of decentralized integration is what many organizations and indeed application teams have always wanted to do, but they may have had to overcome certain technological barriers first.

Note that this move toward decentralized ownership of the integrations and exposure does not necessarily imply a decentralized infrastructure. While each application team clearly could have its own exposure gateways and container orchestration platforms, this is not a given. The important thing is that they can work autonomously. API management is very commonly implemented in this way: with a shared infrastructure (an HA pair of gateways and a single installation of the API management components) yet with each application team directly administering their own APIs as if they had their own individual infrastructure. IBM API Connect, for example, enables this flexibility in approaches where the API infrastructure can be shared in a multi-tenant way, yet it still gives teams full autonomy over the APIs they expose. The same can be done with the integration runtimes by having a centralized container orchestration platform on which they can be deployed, but giving application teams the ability to deploy their own containers independently of other teams.

If there is a downside to this approach, it may be the question of how to govern the multitude of different ways that each application team might use the technology—how to encourage standard patterns of use and best practices. Autonomy can lead to divergence. If every application team creates APIs in their own style and convention, it can become complex for consumers who want to re-use those APIs. With SOA, attempts were made to create rigid standards for every aspect of how the SOAP protocol would be used, which inevitably made them harder to understand and reduced adoption. With RESTful APIs, it is more common to see convergence on conventions rather than hard standards. Either way, the need is clear: Even in decentralized environments, you still need to find ways to ensure an appropriate level of commonality across the enterprise. Of course, if you are already exploring a microservice-based approach elsewhere in your enterprise, then you will be familiar with the challenges of autonomy.

It is worth noting that this decentralized approach is particularly powerful when moving to the cloud. Integration is already implemented in a cloud-friendly way and aligned with the systems

of record. Integrations relating to the application have been separated out from other unrelated integrations so they can move cleanly with the application. Furthermore, container-based infrastructures, if designed using cloud-ready principles and an infrastructure-as-code approach, are much more portable to cloud and make better use of cloud-based scaling and cost models. With the integration also owned by the application team, it can be effectively packaged as part of the application itself. In short, decentralized integration significantly improves your cloud readiness.

We are a now very long way from the centralized ESB pattern—indeed, the term makes no sense in relation to this fully decentralized pattern—but we're still achieving the same intent of making application data and functions available for re-use by other applications across and indeed beyond the enterprise.

What's the difference between "application" and "integration?"

Once the application development group has taken on the integration, there's still an elephant in the room: At what point are they doing *integration*, as opposed to *application*, development?

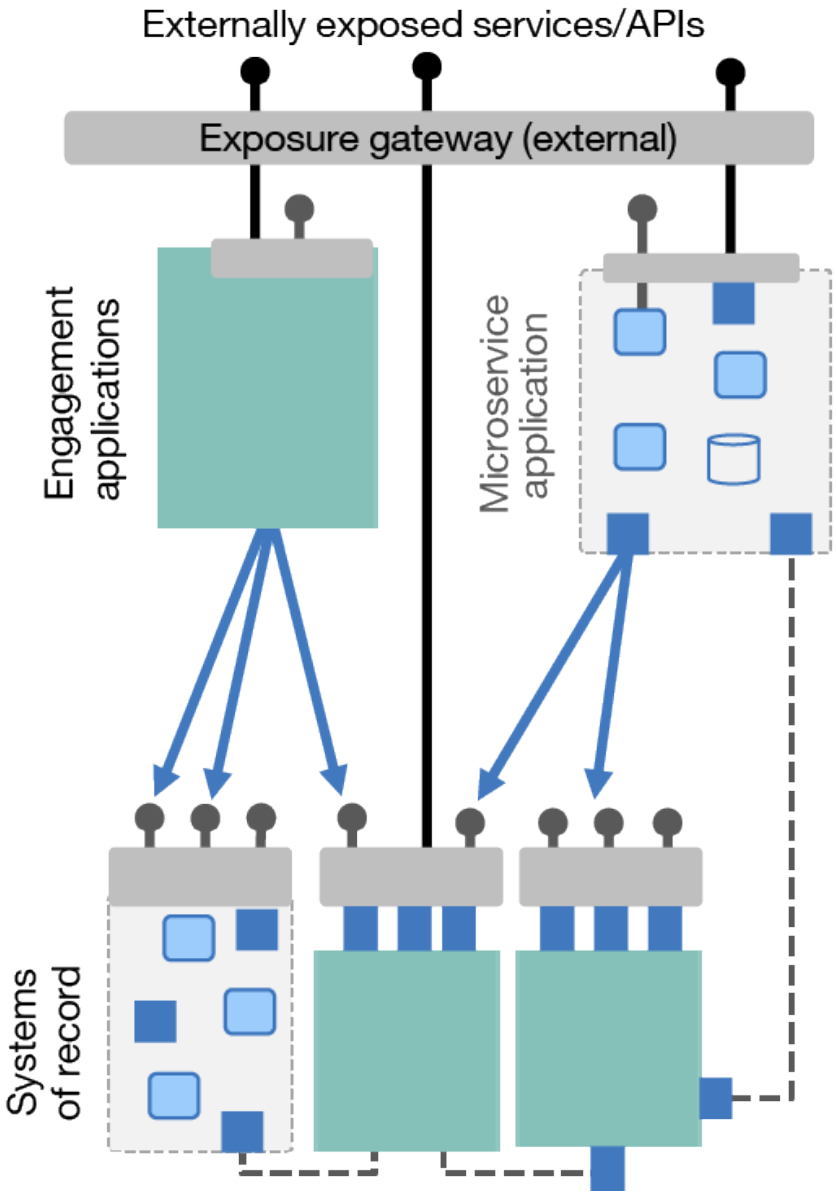
This balancing act between "application" and "integration" constantly dogged SOA, as it deepened the rift between application and integration departments, and the resulting cascade of waterfall-style requirements between the teams slowed projects down. Integration teams were often told they should only do integration logic, not application logic, usually in order to avoid spreading business logic across different teams and components throughout the enterprise.

But what if there weren't separate teams? What if, as in the previous section, it was a single team, with a number of different, specialized tools? For example, one might be focused on the user interface and another might be more focused on a particular need of the solution, such as a rules engine or machine learning. And of course, there is always a need for some integration, too.

In a microservices application, the individual microservice components are relatively small and specialized. It is common to find some whose responsibilities would be well suited to an integration engine. For example, what if all a microservice component did was to provide an API that performed a few invocations to other systems, collated and merged the results, and responded to the caller? That sounds a lot like an *integration*. A simple graphical flow—one that showed which systems you're calling, allowed you to easily find where the data items are merged, and provided a visual representation of the mapping—would be much easier to maintain in the future than hundreds of lines of code.

So, what does this actually mean? As you saw in previous sections, the integration runtime is now a truly lightweight component that can be run in a cloud-native style. One of the key benefits of microservice architecture is that you are no longer restricted to one language or runtime, which means you can have a **polyglot runtime**—a collection of different runtimes, each suited to different purposes. So, there's no technical reason why you can't introduce integration as just another of the runtime options for your microservices applications.

Figure 3. Using a lightweight integration runtime as a microservice component



Let's look at another example. There's a resurgence of interest in messaging in the microservices world, through the popularity of patterns such as event-sourced applications and the use of eventual-consistency techniques. So, you'll probably find plenty of microservice components that do little more than take messages from a queue or topic, do a little translation, and then push the result into a data store. However, they may require a surprisingly large number of lines of code to do it. An integration engine could perform that with easily configurable connectors and graphical data mapping, so you wouldn't have to understand too much about the subtleties of the messaging and data store connectivity. This could also provide an easily maintainable flow diagram and data model mapping for easier maintenance later.

When discussing this approach, an inevitable question is *Am I using an ESB within a microservices application?* It is extremely important to tackle this concern head on. As you may

recall from the earlier definitions, an integration runtime is *not* an ESB. That is just one of the architectural patterns the integration runtime can be a part of. ESB is the heavily centralized, enterprise-scope architectural pattern discussed in Part 1. Using a modern lightweight integration runtime to implement integration-related aspects of an application, deploying each integration independently in a separate component is very different indeed from the centralized ESB pattern. So the answer is *no*, by using a lightweight integration runtime to containerize discrete integrations you are most certainly *not* re-creating the ESB pattern within your microservices application.

This is a radically different interpretation of the purpose of an integration runtime than what it was originally intended to do. Traditionally, integration runtimes have been mostly used for integration *between* separate applications—and they will certainly continue to perform that role—but here we are discussing its use as a component *within* an application.

In the past, this would have been difficult since the integration runtime wasn't seen as part of the application developer's toolbox. Applications were written from the ground up as a single language silo application such as Java on a JEE application server, and that team didn't have the skills to create or run integrations. At that time, the integration runtime was owned and implemented by a completely separate team that had deep skills in the integration product and in integration patterns.

With advances in simplicity of integration runtimes and tooling, there is no longer a need for a separate dedicated team to implement and operate them. Integrations are vastly easier to create and maintain. They can be monitored, administered, scaled, and diagnosed using the same tools and techniques that are used for other microservices components—from container orchestration to routing frameworks, source code repositories, build pipelines, testing tools, and more.

A key benefit of microservices architecture is the freedom it offers you to choose the best runtime for the task at hand. Based on the information above, it is clear that since a microservice-based application has components that are performing integration-like work, there is no reason why a lightweight integration runtime could not be used.

What path should you take?

So far, you have seen how the centralized ESB pattern is in some cases being replaced by the following new approaches:

- **Fine-grained integration deployment** makes use of containerization to enable a much more agile, scalable, and resilient usage of integration runtimes.
- **Decentralized integration ownership** puts the ownership of integrations into the hands of application teams, reducing the number of teams and touchpoints involved in the creation and operation of end-to-end solutions.
- **Cloud-native integration infrastructure** takes advantage of the polyglot nature of microservices architecture to more productively implement integration-focused microservice components by using lightweight integration runtimes.

Each of these approaches is an independent architectural decision that may or may not be right for your organization right now. Furthermore, although this article has described a likely sequence for how these approaches might be introduced, other sequences are perfectly valid.

For example, *decentralized integration ownership* could precede the move to *fine-grained integration deployment* if an organization were to enable each application team to implement their own separate ESB pattern. If we were being pedantic, this would really be an *application service bus* or a *domain service bus*. This would certainly be *decentralized integration ownership*—application teams would take ownership of their own integrations—but it would not be *fine-grained integration deployment* because they would still have one large installation containing all the integrations for their application.

The reality is that you will probably see hybrid integration architectures that use multiple approaches. For example, an organization might have already built a centralized ESB for integrations that are now relatively stable and would gain no immediate business benefit by refactoring. In parallel, they might start exploring the use of containerized lightweight integration runtimes for new integrations that are expected to change quite a bit in the near term.

A final thought: Have we returned to point-to-point?

Comparing the original point-to-point diagram in Part 1 with the final fully decentralized diagram here in Part 2 (Figure 3), it might be tempting to conclude that we have come full circle, and are returning to point-to-point integration. The applications that require data now appear to go directly to the provider applications. Are we back where we started?

To solve this conundrum, you need to go back to what the perceived problem was with point-to-point integration in the first place: Interfacing protocols were many and varied, and application platforms didn't have the necessary technical integration capabilities out of the box. For each and every integration between two applications, you would have to write new, complex, integration-centric code in both the service consumer and the service provider.

Now compare that situation to the modern, decentralized integration pattern. The interface protocols in use have been simplified and rationalised such that many provider applications now offer RESTful APIs— or at least web services—and most consumers are well equipped to make requests based on those standards.

Where applications are unable to provide an interface over those protocols, powerful integration tools are available *to the application teams* to enable them to rapidly expose APIs/services using primarily simple configuration and minimal custom code. Along with wide-ranging connectivity capabilities to both old and new data sources and platforms, these integration tools also fulfill common integration needs such as data mapping, parsing/serialisation, dynamic routing, resilience patterns, encryption/decryption, traffic management, security model switching, identity propagation, and much more—again, all primarily through simple configuration, which further reduces the need for complex custom code.

The icing on the cake is that thanks to the maturity of API management tooling, you are now able to not only expose those interfaces to consumers, but also:

- make them easily discoverable by potential consumers
- enable secure, self-administered on-boarding of new consumers

- provide analytics in order to understand usage and dependencies
- promote them to external exposure so they can be used by external parties
- potentially even monetize APIs, treating them as a product that's provided by your enterprise rather than just a technical interface

So in this more standards-based, API-led integration, there is little burden on either side when a consuming application wants to make use of APIs exposed by another provider application.

Conclusion

So let's briefly review of some examples of the different ways you could explore an agile integration architecture approach given the advances in technology:

- Installing a modern integration runtime takes a handful of minutes at most. Indeed, if using a pre-built Docker image, it could take no more than the few seconds it takes to start up a Docker container.
- Administering a cluster of integration runtimes no longer requires proprietary knowledge, because they would be running in a standard container orchestration platform such as Kubernetes or Docker Swarm.
- Scaling up and down, and setting up resilience policies, routing policies, and secure communication channels would be done in a standardized way, the same as all of your other application components.
- Command-line interfaces and template scripts are provided to enable standard continuous delivery tools that can be used to automate build pipelines. This further reduces the knowledge required to implement interfaces and increases the pace of delivery.
- Integration tooling has improved to the point where most interfaces can be built by configuration alone, often by individuals with no integration background. With the addition of templates for common integration patterns, integration best practices are burned into the tooling, further simplifying the tasks. Deep integration specialists are less often required, and some integration can potentially be taken on by application teams.

So we have clearly come a long way from the old days of point-to-point. Application teams can be empowered with role-appropriate integration capabilities that enable them to expose and consume data across and beyond the enterprise in a way that, if done correctly, is agile, scalable, resilient, and maintainable.

Acknowledgements

Thank you to the following people for their input and review of the material in this article: Andy Garratt, Nick Glowacki, Rob Nicholson, Brian Petrini, and Claudio Tagliabue.

Related topics

- The benefits of agile integration, Part 1: The fate of the ESB
- Lightweight, agile integration: Useful links
- IBM Integration blog
- Try IBM Cloud for free

© Copyright IBM Corporation 2018
(www.ibm.com/legal/copytrade.shtml)
Trademarks
(www.ibm.com/developerworks/ibm/trademarks/)